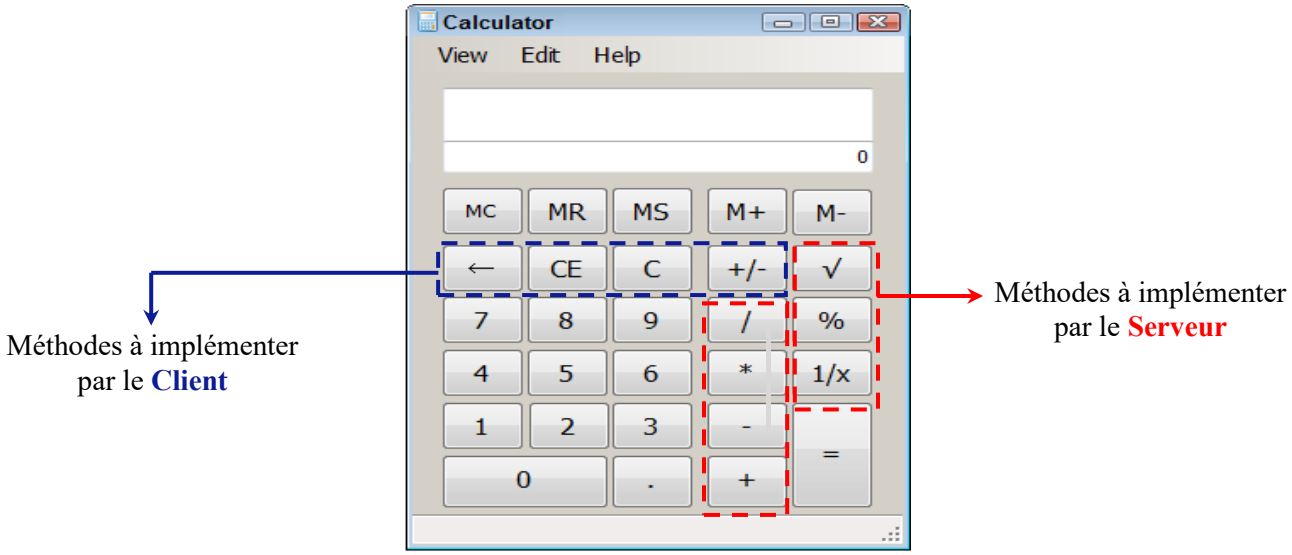
.NetRemoting – TP3 –

I/ Objectif du TP

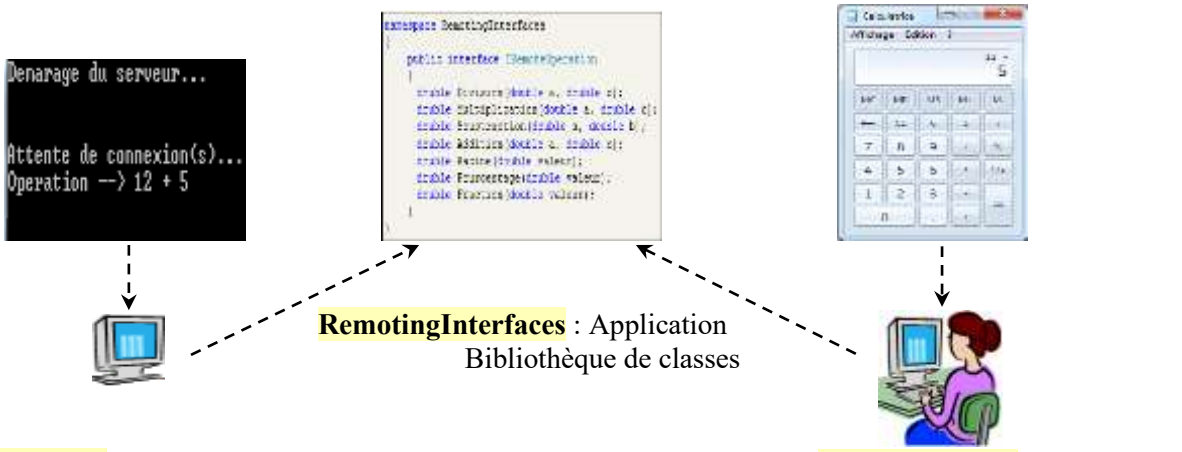
Ce TP aborde dans sa première partie, .NetRemoting qui est la solution Microsoft pour l'appel de méthode à distance, proche de Java RMI (Remote Method Invocation) et la réalisation d'une application **Windows Forms répartie** basée **.NetRemoting** pour la gestion d'un système d'information, dans sa deuxième partie.

II/ Partie 1: Tutorial : Réalisation d'une calculatrice standard distante

Nous allons réaliser une calculatrice standard distante afin de mettre en œuvre le middleware : **.NET Remoting**.



Remarque : le type du client c'est un **Client lourd** car il implémente lui-même quelques méthodes.



RemotingInterfaces : Application Bibliothèque de classes

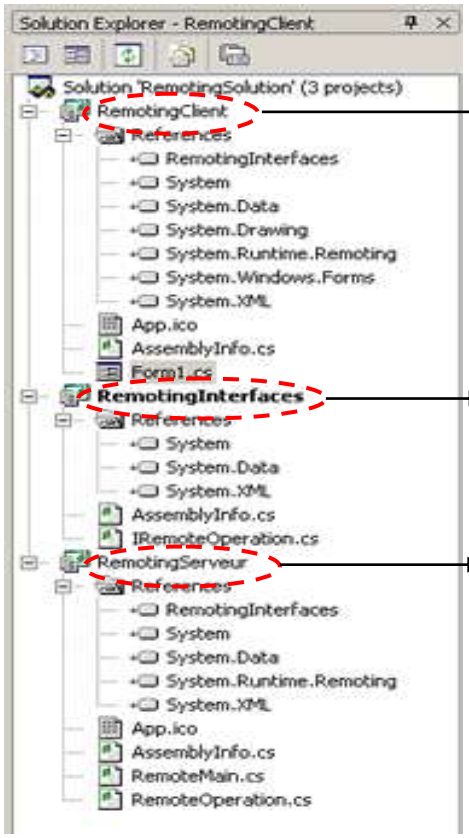
RemotingServeur : Application Console

RemotingClient : Application Windows Forms

.NetRemoting – TP3 –

L'objectif est qu'un serveur offre les fonctions d'une calculatrice standard à travers des méthodes invocables à distance, telle additionner deux entiers, calculer la racine carrée ou le pourcentage d'un entier, etc. Ainsi un Client peut invoquer l'une de ces méthodes distantes en utilisant .NetRemoting.

Pour cela, Créer trois **projets** C# sous Visual studio, qu'on nommera `RemotingClient`, `RemotingInterfaces` et `RemotingServeur`. La structure de la solution finale est donnée ci-dessous :



3. Projet `RemotingClient` : contiendra la classe client qui invoquera l'objet distant (Serveur).

1. Projet RemotingInterfaces : contient la signature des méthodes de l'objet distant à invoquer.

2. Projet `RemotingServeur` : possèdera deux classes l'une qui contient l'implémentation du serveur et l'autre, la méthode main pour lancer le serveur.

1. Projet `RemotingInterfaces`

Pour qu'un client puisse connaître et invoquer des méthodes distantes, il faut:

1. Une `interface` contenant leur déclaration doit être déclarée et référencée côté `client` et côté `serveur`.
2. Le serveur devra implémenter ensuite cette interface.

.NetRemoting – TP3 –

Pour ce faire :

- Ouvrez Visual Studio .NET
- Créez une nouvelle solution vierge, puis nommez la `RemotingSolution`
- Ajoutez à la solution un nouveau projet C# de type "`Librairie de classes`", nommez le : `RemotingInterfaces`.
- Renommez le fichier `Class1.cs` par **`IRemoteOperation`**`.cs`
- Remplacez le code de la classe `Class1` par le code suivant qui déclare la signature des méthodes invocables à distance `Division`, `Multiplication`, `Soustraction`, `Addition`, `Racine`, `Pourcentage` et `Fraction` :
- Compilez le projet, pour cela cliquez avec le bouton droit sur le nom du projet (soit `RemotingInterfaces`) puis cliquer sur `générer`.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace RemotingInterfaces
{
    public interface IRemoteOperation
    {
        double Division(double a, double b);
        double Multiplication(double a, double b);
        double Soustraction(double a, double b);
        double Addition(double a, double b);
        double Racine(double valeur);
        double Pourcentage(double valeur);
        double Fraction(double valeur);
    }
}
```

L'interface est un élément commun entre le `serveur` et le `client`. C'est un projet de type "`Librairie de classes`" dont la compilation générera l'interface dans une librairie `dll` pour simplifier la distribution côté `Client` et `Serveur`.

- Pour vérifier l'existence du fichier dll, cliquer :
  1. Avec le bouton droit sur le nom du projet (soit `RemotingInterfaces`),
  2. Sur "`Ouvrir le dossier dans l'explorateur windows`",

.NetRemoting – TP3 –

3. Une fois l'explorateur est ouvert, accéder au répertoire `bin\Debug` (ou `bin\Release`) et vous remarquez que la compilation du projet, a créé un fichier `dll`.

2. Projet `RemotingServeur`

Le serveur implémente l'interface `IRemoteOperation`, ainsi son code devra être composé par étape comme suit :

- Ajouter à la solution `RemotingSolution` créée précédemment, un nouveau projet C# de type `Application Console`, nommez le `RemotingServeur`.
- Afin que le serveur implémente l'interface créée, deux possibilités sont possibles :
  1. Copier le fichier `IRomoteOperation.cs` dans le projet `RemotingServeur`,
  2. Référencer le fichier `dll` qui contient l'interface.

Dans notre cas, nous allons référencer le fichier `dll`. Pour cela cliquer :

1. Avec bouton droit de la souris sur le projet,
2. Sur `ajouter une référence`, l'onglet `projet`,
3. Sur `RemotingInterfaces`.

Ce projet contiendra deux classes :

1. **`RemoteOperation`**`.cs` contenant l'implémentation de l'interface,
2. **`RemoteMain`**`.cs` contenant la méthode `main` qui lance le serveur.

- Ajouter au projet courant une classe `RemoteOperation.cs`
- Mettez le code suivant dans la classe créé, il s'agit de l'objet distant :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace RemotingServeur {
    public class RemoteOperation : MarshalByRefObject,
                  RemotingInterfaces.IRemoteOperation {

      private double nombre = 0;
      /*-------------------------------------------------------------------*/
      // Indique que l'objet aura une durée de vie illimitée
```

.NetRemoting – TP3 –

```
        public override object InitializeLifetimeService() {
            return null;
        }
        /*-------------------------------------------------------------------*/
        // Définition de la Division
        public double Division (double a, double b) {
            Console.WriteLine("Appel client de la méthode Division : a="+a+" b="+b);
            if (b!=0) return a / b;
            else return -1;
        }
        /*-------------------------------------------------------------------*/
        // Définition de la Multiplication
        public double Multiplication(double a, double b)
        {
            Console.WriteLine("Appel client de la méthode Multiplication : a=" + a + " b=" +
                                b);
            return a * b;
        }
        /*-------------------------------------------------------------------*/
        // Définition de la Soustraction
        public double Soustraction(double a, double b)
        {
             Console.WriteLine("Appel client de la méthode Soustraction : a=" + a + " b=" + b);
            return a - b;
        }
        /*-------------------------------------------------------------------*/
        // Définition de l'Addition
        public double Addition(double a, double b) {
            Console.WriteLine("Appel client de la méthode Addition : a=" + a + " b=" + b);
            return a + b;
        }
        /*-------------------------------------------------------------------*/
        // Définition de la Racine
        public double Racine(double valeur) {
            Console.WriteLine("Appel client de la méthode Racine : valeur=" + valeur);
            return Math.Sqrt(valeur);
        }
        /*-------------------------------------------------------------------*/
        // Définition de la Pourcentage
        public double Pourcentage (double valeur) {
            Console.WriteLine("Appel client de la méthode Pourcentage : valeur=" + valeur);
            return valeur / 100 ;
        }
        /*-------------------------------------------------------------------*/
        // Définition de la Fraction
        public double Fraction (double valeur) {
            Console.WriteLine("Appel client de la méthode Fraction : valeur=" + valeur);
            if (valeur!=0) return 1 / valeur ;
            else return -1;
        }
        /*-------------------------------------------------------------------*/
    }
}
```

.NetRemoting – TP3 –

- `RemoteOperation` implémente la classe `MarshalByRefObject` qui indique que le client ne contiendra qu'une référence de l'objet et non pas l'objet lui-même (une sorte de pointeur distant de fonction).
- Ce fonctionnement est à l'opposé à `MarshalByValue` où une copie complète d'un objet sérialisable est passée au client (non utiliser dans ce TP).
- Les méthodes `Division`, `Multiplication`, `Soustraction`, `Addition`, `Racine`, `Pourcentage` et `Fraction` sont aussi définies conformément à l'interface `IRemoteOperation`.
- `InitializeLifetimeService` est surchargée en retournant `null`. Sans cela, la durée de vie de l'objet serait par défaut : au premier appel : 5 min puis de 2 min à chaque appel. Elle est, dans notre cas, infinie. La possibilité de durée de vie limitée peut être intéressante mais, dans le cadre de ce tutorial.

Configuration et lancement du serveur

Ajoutez une référence à `System.Runtime.Remoting`, pour ce faire cliquer :

1. Avec bouton droit de la souris sur le projet
2. Avec bouton droit de la souris sur le projet,
3. Sur `ajouter une référence`, l'onglet `.NET`,
4. Sur `System.Runtime.Remoting`.

- Renommez le fichier `Class1.cs` par `RemoteMain.cs`
- Placez-y le code suivant, il permet de démarrer le serveur :
- Une fois le code ajouté, compilez le projet "`RemotingServeur`".

```
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System;
using System.Runtime.Remoting;
using System.Runtime.Remoting.Channels;
using System.Runtime.Remoting.Channels.Tcp;

namespace RemotingServeur
{
    public class RemoteMain
    {
        [STAThread]
        public static void Main(string[] args)
```

.NetRemoting – TP3 –

```
        {
            try
            {
                // Création d'un nouveau canal d'écoute sur le port 1069
                TcpChannel channel = new TcpChannel(1069);



                // Enregistrement du canal
                ChannelServices.RegisterChannel(channel, true);  // false : en cas de pb
                // Démarrage de l'écoute en exposant l'objet en Singleton
                RemotingConfiguration.RegisterWellKnownServiceType(
                                        typeof(RemoteOperation), "RemoteOperation",
                                        WellKnownObjectMode.Singleton);
                Console.WriteLine("Le serveur a démarré avec succès");
                Console.ReadLine();
            }
            catch
            {
                Console.WriteLine("Erreur lors du démarrage du serveur");
                Console.ReadLine();
            }
        }
    }
}
```

Nous créons un nouveau `Channel` (canal) de type Tcp. Des `channels` d'autres types pourraient être utilisés comme `HttpChannel` ou des développements non standards comme `IIOPChannel`. `TcpChannel` a l'avantage ici d'être plus rapide que `HttpChannel` puisque les données transitent en binaire. Ce `channel` est ensuite enregistré sur le `serveur` pour être trouvé par le `client`, puis l'écoute est lancée avec l'objet `RemoteOperation` exposé en mode `Singleton`.

Deux types de modes d'***activation côté serveur*** peuvent être accordés à un objet distant avec la technologie .Net Remoting :

1. ***Singleton*** : Un seul objet est partagé entre les clients et les appels pendant une durée de vie fixée. Lorsque la durée de vie est expirée, l'objet est ré-instancié.
2. ***SingleCall*** : A chaque appel, le serveur crée une nouvelle instance de l'objet qui est détruit après chaque utilisation.

Sachez également qu'il est possible, en passant par une ***activation côté client*** de l'objet, que le serveur crée une instance de l'objet pour chaque client.

.NetRemoting – TP3 –

Remarque : Nous avons utilisé `RegisterWellKnownServiceType()`. Or, il existe une autre méthode de la classe **`RemotingConfiguration`** : `Configure()`. Cette méthode permet de charger un fichier `xml` contenant toutes les informations de configuration du serveur distant. Cette alternative peut s'avérer très pratique même si nous ne l'avons pas utilisée ici.

3. Projet `RemotingClient`

Le but de ce projet est de créer une classe Client qui cherche l'objet distant, puis invoque ses méthodes distantes. Pour cela, suivez les étapes suivantes :

- Ajoutez à la solution créée précédemment un nouveau projet C# de type `Application Windows Forms`, nommez le `RemotingClient`
- Renommer la classe `Form1.cs` par `Client.cs`
- Ajoutez une référence au projet `RemotingInterfaces`
- Ajoutez une référence à `System.Runtime.Remoting`
- Concevez à l'aide de la boîte à outils l'interface de la calculatrice. pour cela ajouter sur `Form1.cs[Design]` les composants suivant : `Button` (28), `TextBox` (2), `MenuStrip` (1), `StatusStrip` (1). Modifier les propriétés des composants selon le besoin pour réaliser l'interface de la calculatrice.
- Mettez le code suivant dans la classe `Client`
- Reliez chaque méthode du code à son composant correspondant. Pour cela modifier la propriété Events avec le nom de la méthode adéquate.
- Compilez le projet

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.Runtime.Remoting;
using System.Runtime.Remoting.Channels;
using System.Runtime.Remoting.Channels.Tcp;
```

.NetRemoting – TP3 –

```
namespace RemotingClient
{
    public partial class Client : Form
    {
        private bool First = true;

        private string Chaine, Operateur, Quotient;
        private double Operande1, Operande2;

        RemotingInterfaces.IRemoteOperation remoteOperation;

        //_____________________________________________________________________//
        public Client()
        {
            InitializeComponent();

            try
            {
                TcpChannel channel = new TcpChannel();
                ChannelServices.RegisterChannel(channel, true);  // false : en cas de pb
                remoteOperation =
                    (RemotingInterfaces.IRemoteOperation)Activator.GetObject(
                    typeof(RemotingInterfaces.IRemoteOperation),
                    "tcp://localhost:1069/RemoteOperation");
            }
            catch { statusStrip.Text = "Erreur de connexion au serveur"; }
            //-------------------------------------------------------------------
        }
        //_____________________________________________________________________//
        //            Méthode Click sur n'importe quel bouton (0 à 9)          //
        //_____________________________________________________________________//
        private void Button_Click(object sender, EventArgs e)
        {
            if (this.Operation.Text == "0") { this.Operation.Clear(); }
            this.Operation.Text = this.Operation.Text + ((Button)sender).Text;

            if (this.Resultat.Text == "") { this.Resultat.Text = "("; }
            this.Resultat.Text = this.Resultat.Text + ((Button)sender).Text;
        }
        //_____________________________________________________________________//


        /*-------------------------------------------------------------------*/
        /*         Les Méthodes réalisées par le client (Client lourd)       */
        /*-------------------------------------------------------------------*/
        private void PlusMoins_Click(object sender, EventArgs e)
        {
            this.Operation.Text = (-(int.Parse(this.Operation.Text))).ToString();
            if (First)
            {
                this.Resultat.Text = "(" + this.Operation.Text;
                Chaine = this.Resultat.Text;
            }
            else
```

.NetRemoting – TP3 –

```
            {
                if (int.Parse(this.Operation.Text) < 0)
                    this.Resultat.Text = Chaine + "(" + this.Operation.Text + ")";
                else this.Resultat.Text = Chaine + this.Operation.Text;
            }
        }
        //_____________________________________________________________________//
        private void C_Click(object sender, EventArgs e)
        {
            Operation.Clear();
        }
        //_____________________________________________________________________//
        private void CE_Click(object sender, EventArgs e)
        {
            Operation.Text = "0"; Resultat.Clear();
        }
        //_____________________________________________________________________//
        private void Retour_Click(object sender, EventArgs e)
        {
            // this.Resultat.Clear(); this.Resultat.Text = 0.ToString();
            this.Operation.Undo();
        }
        //_____________________________________________________________________//

        /*---------------------------------------------------------------------*/
        /*  Les Méthodes invoquées par le client et réalisées par le serveur   */
        /*---------------------------------------------------------------------*/
        private void Operation_Click(object sender, EventArgs e)
        {
            if (this.Resultat.Text != "")
            {
                First = false; this.Operande1 = int.Parse(this.Operation.Text);
                Operateur = ((Button)sender).Text;
                this.Operation.Clear();
                this.Resultat.Text = this.Resultat.Text + ((Button)sender).Text;
                Chaine = this.Resultat.Text;
            }

            else this.Resultat.Text = ((Button)sender).Text;
        }

        //_____________________________________________________________________//
        private void Egal_Click(object sender, EventArgs e)
        {
            if (this.Resultat.Text != "")
            {
                this.Resultat.Text = this.Resultat.Text + ")=";
                this.Operande2 = int.Parse(this.Operation.Text);
            }


        //_____________________________________________________________________//
        /* Appel des méthodes qui seront exécutées par le Serveur              */
        /* Le résultat sera affiché par le Client                              */
```

.NetRemoting – TP3 –

```
        //_____________________________________________________________________//
            if (remoteOperation != null)
            {
                statusStrip.Items[0].Text = "Connexion au serveur réussie ..... ";
                switch (this.Operateur)
                {
                    case "/": Quotient = (remoteOperation.Division(this.Operande1,
                              this.Operande2)).ToString();
                        if (int.Parse(Quotient) == -1)
                              { statusStrip.Items[0].Text = "Division impossible ";
                                this.Resultat.Text = this.Resultat.Text + "?"; }
                        else { this.Operation.Text = Quotient;
                               this.Resultat.Text = this.Resultat.Text + this.Operation.Text; }
                        break;

                    case "*": this.Operation.Text =
                              (remoteOperation.Multiplication(this.Operande1,
                              this.Operande2)).ToString();
                              this.Resultat.Text = this.Resultat.Text + this.Operation.Text;
                              break;

                    case "-":  this.Operation.Text =
                               (remoteOperation.Soustraction(this.Operande1,
                               this.Operande2)).ToString();
                               this.Resultat.Text = this.Resultat.Text + this.Operation.Text;
                              break;

                    case "+":  this.Operation.Text = (remoteOperation.Addition(this.Operande1,
                               this.Operande2)).ToString();
                               this.Resultat.Text = this.Resultat.Text + this.Operation.Text;
                               break;

                    case "√": this.statusStrip.Text = "Opérande..." + this.Operande1;
                              this.Operation.Text =
                              (remoteOperation.Racine(this.Operande1)).ToString();
                              // this.Resultat.Text = this.Resultat.Text + this.Operation.Text;
                               break;

                    default: this.statusStrip.Text = "Opération invalide...";
                             break;
                } } }
        //_____________________________________________________________________//
        private void Racine_Click(object sender, EventArgs e)
        {
            First = false; this.Operande1 = int.Parse(this.Operation.Text);
            Operateur = "√";
            this.Resultat.Text = "√" + this.Resultat.Text;
            Chaine = "√(" + this.Operande1 + ")";
        }
        //_____________________________________________________________________//
    } }
```

.NetRemoting – TP3 –

Nous avons déclaré un channel de type Tcp puis nous avons instancié une référence à l'objet distant en indiquant l'interface locale.

Les paramètres de connexion et le nom de l'objet distant implémentant l'interface.

Nous avons récupéré ainsi un objet "remoteOperation" manipulable comme un objet classique mais dont l'implémentation est sur un serveur dissocié.

4. Exécution

- Lancez le serveur (**RemotingServer.exe**)
- Lancez le client (**RemotingClient.exe**)
- Testez une addition simple, si le résultat apparaît, alors votre programme fonctionne.